

A 3D rendering of the text "DL for ENG" in a bold, gold, sans-serif font. The letters are floating on a deep blue ocean with visible ripples and small waves. The sky above is a mix of light blue and grey, with soft, white clouds. The overall scene has a cinematic, high-quality feel.

DL for ENG

LECTURE 5.2

RNN and LSTM

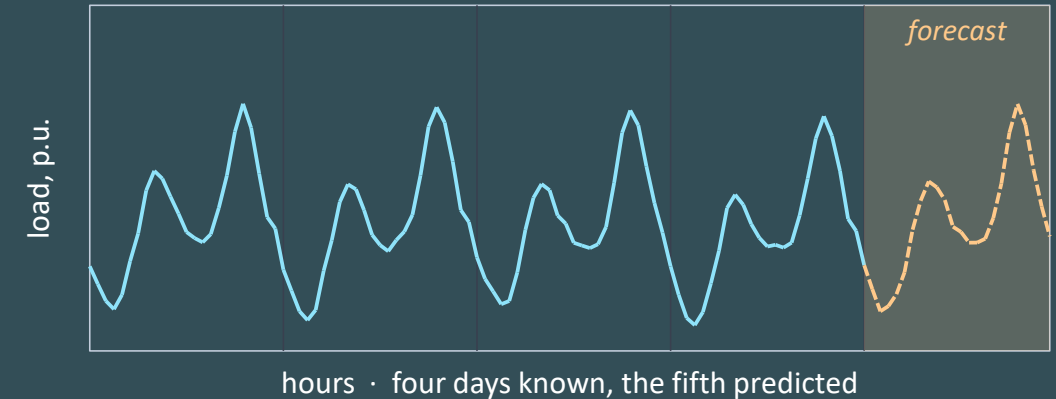
Remus Teodorescu

*With support from Research Assistant Noman Khan -
nomank@energy.aau.dk*

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

SEQUENCES · RNN · LSTM



A recurrent network reads a sequence one step at a time and carries a memory, its hidden state, from each step to the next with the same weights: here from four days of substation demand to the fifth. An LSTM adds a cell state guarded by three gates, so that the memory lasts for days rather than hours.

Learning Objectives

SECTIONS, IN ORDER

- 1 **Sequences** — data in time, and the baseline to beat
- 2 **RNN** — a network with a memory
- 3 **LSTM** — a memory that lasts, and how to use it

READ, AND WHAT EACH ANSWERS

- | | |
|----------------------|--|
| UDL ch. 12.1 | what makes a sequence different, and what must a model beat? |
| GBC ch. 10.1 | how does a network keep a memory? |
| GBC ch. 10.10 | how does a memory last, and how is it tuned? |

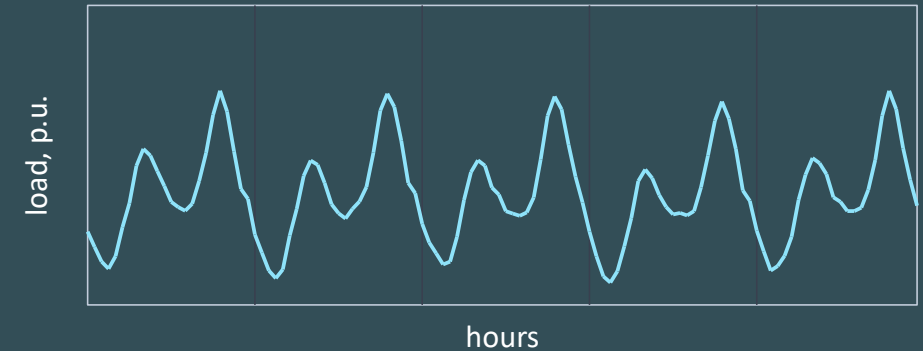
Sequences

Sequence — data whose order carries meaning, such as a load measured every hour. Each value is read together with the ones before it; shuffle the order and the meaning is gone.

DATA IN TIME

- **examples** — grid demand, a sensor trace, a sentence
- **order matters** — shuffle the hours and the pattern is gone
- **the task in notebook 04** — an hourly load forecast
- **persistence** — the next hour equals this hour
- **the baseline to beat** — any model must do better than that

FIVE DAYS OF DEMAND



WHAT A SEQUENCE MODEL NEEDS

read in order	one step after another
remember	what came before

Recurrent Network

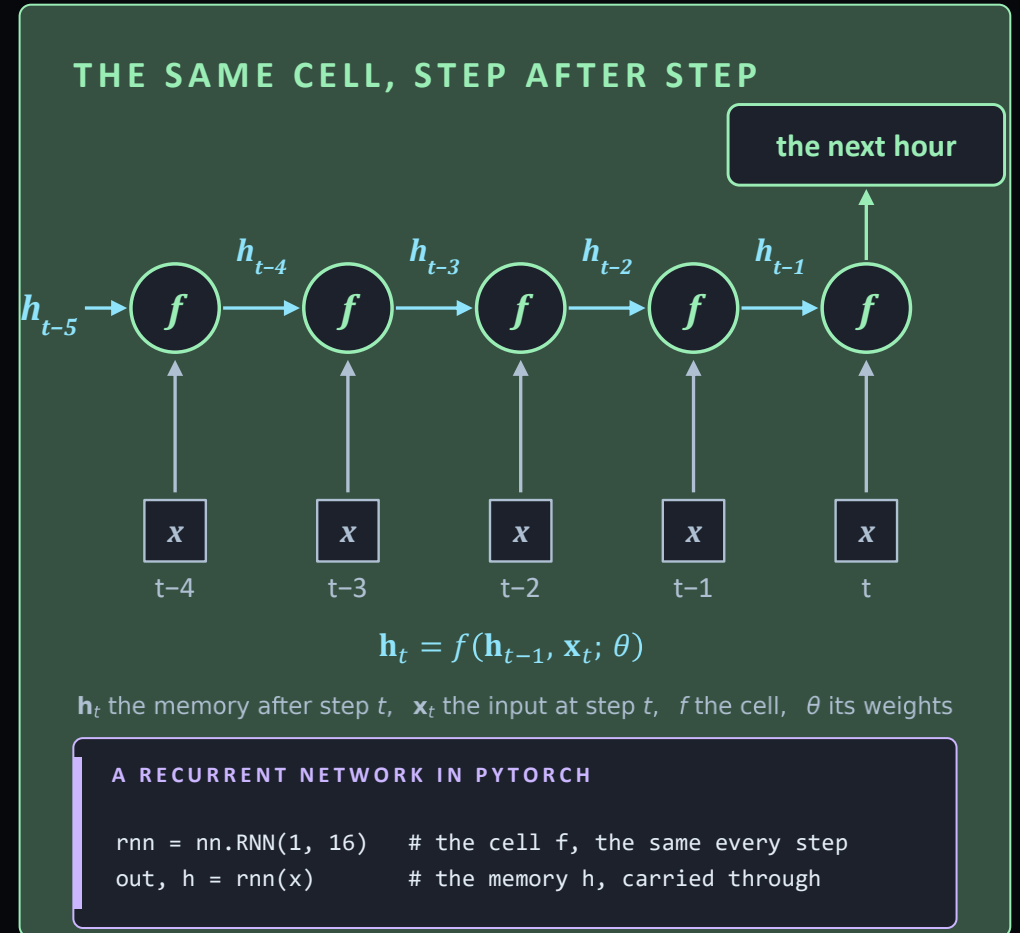
Recurrent network — a network that reads a sequence one step at a time and carries a hidden state from step to step, using the same weights at every step.

KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a network with a memory, fed back at every step
- **function** — a sequence in, one answer at the end
- **architecture** — one cell, applied at every step
- **applications** — load and wind forecasting, control signals, speech

AGAINST A DENSE NETWORK

	dense network	recurrent network
input	one vector, fixed length	a sequence of any length
memory	none between inputs	the hidden state h
weights	one set per input position	one cell, shared over time
parameters	grow with the window	independent of the length
order	an ordinary input index	built in: step t follows $t - 1$



Hidden State

Hidden state — the internal vector $h(t)$ an RNN updates at each time step from the current input and its previous value, acting as a learned memory that summarises the sequence seen so far.

A SUMMARY OF THE PAST, UPDATED EACH STEP

- **the hidden state** — a short summary of everything seen so far
- **each step** — the new input and the state, weighted and summed
- **tanh** — the activation function, squashing into $-1 \dots 1$

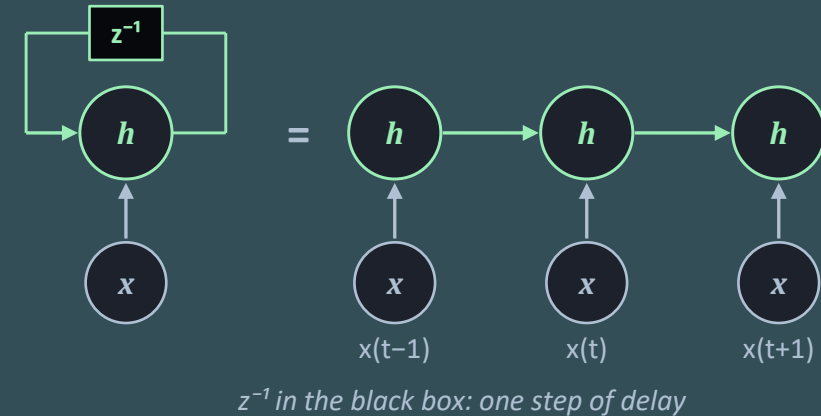
$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

$$\hat{y} = \mathbf{W}_y \mathbf{h}_T + b_y$$

$\mathbf{x}_t$ the input at step t , $\mathbf{h}_t$ the state, $\mathbf{W}_h, \mathbf{W}_x$ shared every step, $\hat{y}$ read from the last state

TALK · 2 MIN In your own words, why can one set of weights read a sequence of any length? Then, in the unfolded chain on the right, which inputs has $h(t+1)$ seen, and along which arrows did they reach it?

FOLDED AND UNFOLDED



THE SAME, AS A LOOP

```
h = h0                                # start: an empty memory
for t in 1 ... T:                      # one step per hour
    h = tanh(Wx@x[t] + Wh@h + b)      # new memory
    y = Wy @ h + by                  # read out the forecast
```

W_x reads the input, W_h reads the memory — two jobs, two matrices, both the same at every step

LSTM

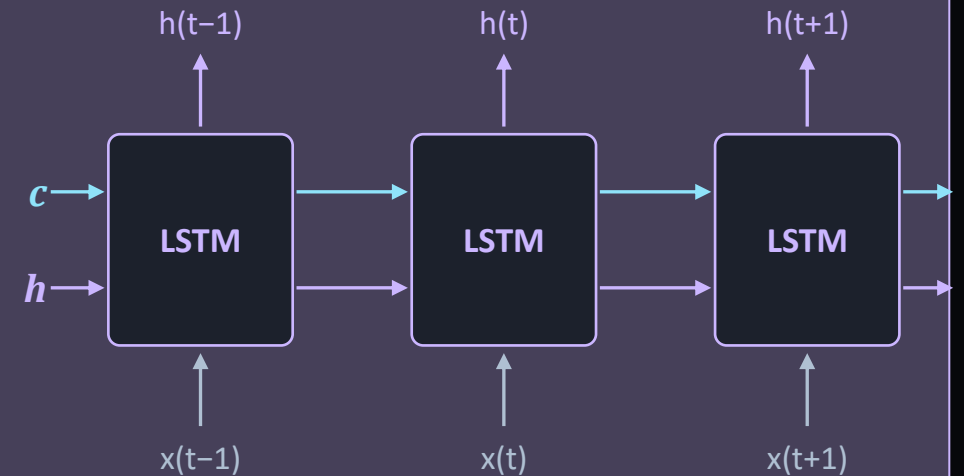
LSTM — keeps two memories: the cell state, a long-term memory carried along the whole sequence and changed only slightly at each step, and the hidden state, a short-term working memory that reads out the parts of the long-term memory relevant right now and passes them to the next step and to the output.

KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a recurrent network with a longer memory
- **function** — a sequence in, an output per step or at the end
- **architecture** — a cell state and three gates that control it
- **applications** — forecasting, speech, text, state estimation

	plain recurrence	LSTM
memory	one vector h	h , and a cell state c beside it
each step	h is rewritten	c is kept, then added to
gates	none	forget, input, output
how long	short	much longer
parameters	one set	four sets, four times as many

AN LSTM LAYER, UNROLLED

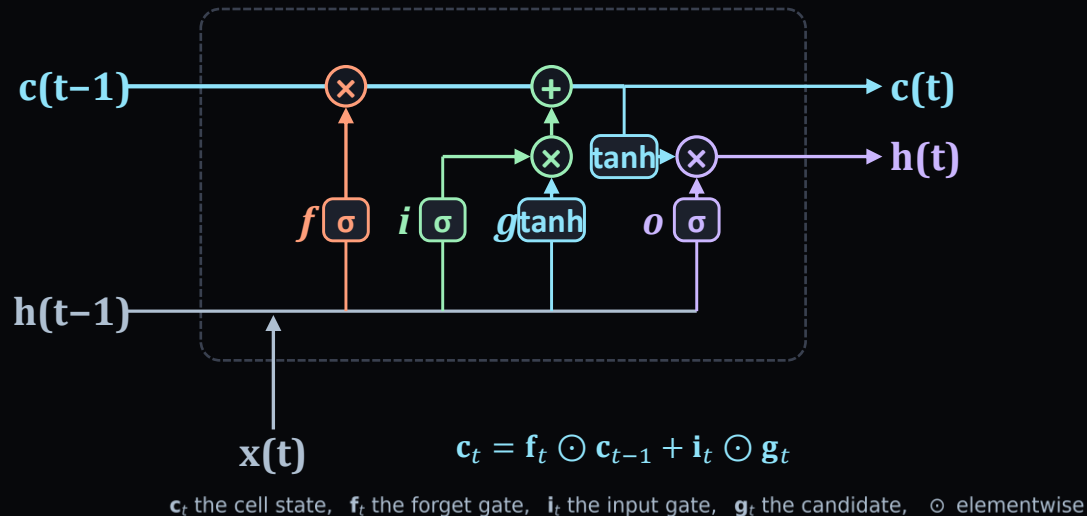


two memories cross every step · c by addition, h as the output

LSTM Cell

LSTM cell — a recurrent unit with a cell state controlled by three learned gates: the forget gate decides how much of the previous cell state to keep, the input gate how much new information to write in, and the output gate which parts of the cell state to expose as the hidden state. This additive memory path preserves gradients over long sequences.

A BELT, AND THREE GATES THAT CONTROL IT



the belt is kept and added to, not rewritten: the memory lasts

THE SIX EQUATIONS, GATE BY GATE

f $f_t = \sigma(W_f[x_t, h_{t-1}] + b_f)$

forget gate — how much of the belt to keep

i $i_t = \sigma(W_i[x_t, h_{t-1}] + b_i)$

input gate — how much new to add

g $g_t = \tanh(W_g[x_t, h_{t-1}] + b_g)$

candidate — new content from x and h, not x itself

o $o_t = \sigma(W_o[x_t, h_{t-1}] + b_o)$

output gate — how much to show as h

c $c_t = f_t \odot c_{t-1} + i_t \odot g_t$

cell update — the line drawn on the left

h $h_t = o_t \odot \tanh(c_t)$

hidden state — the output

THE LSTM CELL IN PYTORCH

```
cell = nn.LSTMCell(1, 16) # the six lines, one call
h, c = cell(x[:, t], (h, c)) # σ gates 0-1, tanh -1-1
```

LSTM Hyperparametrization

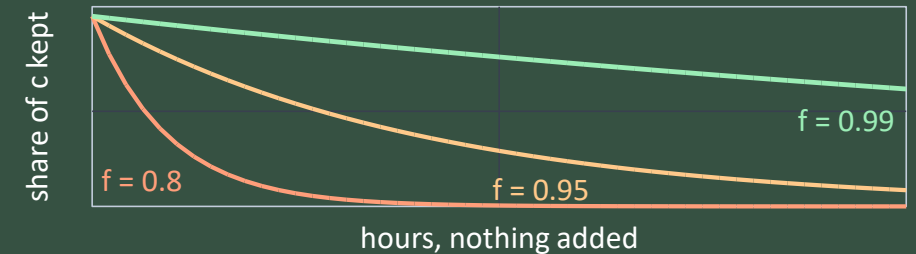
Hyperparameters — an LSTM cell contains input weights, recurrent weights and biases for the forget, input and output gates, each acting as a learned gain between 0 and 1, and for the candidate cell value; these are learned in training. The main hyperparameters are the hidden layer width, the number of stacked layers, and whether to use dropout or bidirectionality.

SIZE, INITIALISE, AND KNOW THE COST

- **hidden width** — start small, 16 here; grow only if it underfits
- **layers** — one, or two at most
- **forget bias at one** — it remembers by default
- **compare** — against persistence, on held-out hours

TALK · 2 MIN In your own words, what does a memory give a forecaster that a fixed window does not? Then, in the plot on the right, after about how many hours has a forget gate of 0.95 kept only half of the cell state?

HOW LONG THE CELL REMEMBERS



computed as f to the k · grid: one half, one day

HYPERPARAMETRIZATION IN PYTORCH

```
lstm = nn.LSTM(input_size=1,      # one value per hour
               hidden_size=16,    # width of h and c
               num_layers=1,      # LSTM layers stacked
               batch_first=True)  # x: (batch, 24, 1)
head = nn.Linear(16, 1)          # dense layer: h -> forecast
out, (h, c) = lstm(x)            # h at every hour, last c
y = head(out[:, -1])             # forecast from the last h
```

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
self-attention	every step looks at every other	UDL ch. 12.2	machine translation
transformer	attention and dense layers stacked	UDL ch. 12.4	large language models
vanishing gradient	why a plain RNN forgets the past	GBC ch. 10.7	long speech and text
gated recurrent unit	a lighter cell: two gates	GBC ch. 10.10	speech recognition
bidirectional network	reads forwards and backwards	GBC ch. 10.3	transcribing speech
sequence to sequence	one network reads, one writes	GBC ch. 10.4	machine translation

WHY THEY ARE LEFT OUT

These pay off on huge collections of text and speech, where the sequences are long and the data plentiful. A substation forecast from 24 hours of load is neither: persistence, a recurrent network or an LSTM is enough.

Exercise 5

NOTEBOOK 04, AND THE REPORT

- **04** — an hourly load forecast: persistence, a recurrent network and an LSTM, each in its own cell
- **compare** — held-out error against persistence, and the forecast plot, where the models differ
- **01 to 03** — from L5.1: the CNN, graphs from scratch, the graph network on the power flow
- **05** — the report: the answers of notebooks 01 to 04, gathered, and one question to conclude

THE QUESTIONS NOTEBOOK 04 ANSWERS

baseline	how good is persistence already?
RNN	does a memory beat it?
LSTM	worth its extra parameters?

REPORT QUESTION, TO CONCLUDE

Across a grid, a graph and a sequence: when did building the data's structure into the network beat a dense network, and what did it cost?

Key Takeaways

RECURRENT NETWORK

what it is

a network that reads a sequence step by step, with a memory

function

a sequence in; one answer at the end, or one per step

train, tune

Adam through the unrolled steps; window, hidden width, layers

vs dense

one set of weights for any length; 321 parameters here

used for

load and wind forecasting, control signals, speech

LSTM

what it is

a recurrent network with two memories: cell state and h

function

three gates decide what to keep, what to add, what to show

train, tune

as the RNN; forget bias at one; width 16, one layer here

vs dense

any length, a longer memory; 1,233 parameters, lowest error

used for

forecasting, speech, text, state estimation

Next — L6.1: loss functions and gradients.

Questions

1 – 5 · RECURRENT NETWORK

- 1 What kind of network is a recurrent network, and what makes a sequence different from a table of rows? *GBC ch. 10; Ex_05 notebook 04*
- 2 What function does it implement: write the hidden-state update and the read-out, and name the activation? *GBC ch. 10.1 – 10.2*
- 3 How is it trained, and what are its hyperparameters: window, hidden width, layers? *GBC ch. 10.2; Ex_05 notebook 04*
- 4 How does it differ from a dense network in input, memory, weights and parameters? *GBC ch. 10.2; Ex_05 notebook 04*
- 5 What is it used for, and what is the persistence baseline it has to beat? *GBC ch. 10; Ex_05 notebook 04*

6 – 10 · LSTM

- 6 What kind of network is an LSTM, and what are its two memories? *GBC ch. 10.10*
- 7 What function does the LSTM cell implement: the three gates, the candidate and the cell update? *GBC ch. 10.10*
- 8 How is an LSTM trained and hyperparametrized, and why set the forget bias to one? *GBC ch. 10.10 – 10.11; Ex_05 notebook 04*
- 9 How do its results differ from a dense and a recurrent network's, and how long does its cell remember? *GBC ch. 10.10; Ex_05 notebook 04*
- 10 What is it used for, which forecast won on held-out error in notebook 04, and what does the plot show? *Ex_05 notebook 04*

NEXT

L6.1 — Loss Functions and Gradients

Every architecture this course uses is now in hand: dense, convolutional, graph, recurrent and LSTM. Next: how they are trained.

BEFORE THE NEXT SESSION

read	UDL ch. 5.1 – 5.3
run	Ex_05 notebooks 04 and 05
bring	your forecast plot
think about	which forecast to deploy, and why

bring your held-out errors from notebook 04